

Support Vector Machine (SVM)

Importing the libraries

```
In [3]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the dataset

```
In [4]: dataset = pd.read_csv(r'C:\Users\sande\OneDrive\Desktop\Naresh IT\data science and dataset')
```

```
Out[4]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

```
In [5]: X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, -1].values
```

Splitting the dataset into the Training set and Test set

```
In [6]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20, random_
```

Feature Scaling

```
In [7]: from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Training the SVM model on the Training set

```
In [8]: from sklearn.svm import SVC
classifier = SVC()
classifier.fit(X_train, y_train)
```

Out[8]:

SVC		
▼ Parameters		
	C	1.0
	kernel	'rbf'
	degree	3
	gamma	'scale'
	coef0	0.0
	shrinking	True
	probability	False
	tol	0.001
	cache_size	200
	class_weight	None
	verbose	False
	max_iter	-1
	decision_function_shape	'ovr'
	break_ties	False
	random_state	None

Predicting the Test set results

```
In [9]: y_pred = classifier.predict(X_test)
```

Making the Confusion Matrix

```
In [11]: from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[55  3]
 [ 1 21]]
```

This is to get the Models Accuracy

```
In [12]: from sklearn.metrics import accuracy_score
ac = accuracy_score(y_test, y_pred)
print(ac)
```

```
0.95
```

```
In [13]: bias = classifier.score(X_train, y_train)
print(bias)
```

```
0.903125
```

```
In [14]: variance = classifier.score(X_test, y_test)
print(variance)
```

```
0.95
```

This is to get the Classification Report

```
In [15]: from sklearn.metrics import classification_report
cr = classification_report(y_test, y_pred)
cr
```

```
Out[15]: '          precision    recall  f1-score   support\n\n 0.95      0.96      0.58         1         0.88      0.95      0.91      22\n\n accuracy          0.95      80\n\n macro avg          0.95      0.95      0.95      80\n\nweighted avg          0.95      0.95      0.95      80'
```

```
In [ ]:
```